

2026 KIST AIX 성과 공유회 · 기술 문서

키스토(KISTO) 바탕화면의 AI 연구 동료

멀티에이전트 연구 파이프라인과 이중 모델 교차검증의 설계·구현

문서 범위	시스템 구조, 핵심 알고리즘 6종, 실측 결과, 보안, 한계
기준 버전	2026-09-23 (저장소 main 브랜치)
저장소	github.com/kihowork94-korea/kisto
시연 데이터	분석 단계의 수치는 시연용 가상 데이터로 계산한 값이다 (실측 연구값 아님)

목차

1	개요 — 문제 정의와 설계 원칙
2	시스템 구조 — '나만의 연구실'
3	알고리즘 1 · 의도 라우팅과 상태 기반 모듈 선택
4	알고리즘 2 · 이중 모델 교차검증 배정과 자동 대체
5	알고리즘 3 · 근거 고정 인용 (Grounded Citation)
6	알고리즘 4 · 자가 수정 분석 루프
7	알고리즘 5 · 대필하지 않는 집필 프로토콜
8	알고리즘 6 · 연구 메모리와 실데이터 기반 게임화
9	실측 결과
10	보안·개인정보, 한계와 향후 과제

1 개요

1.1 문제 정의

도메인 경험이 부족한 신입 연구자는 선행 연구 파악·가설 설정·분석 해석·논문 서론 작성에서 오래 막힌다. 범용 AI 챗은 도움이 되지만 다음 네 가지가 남는다.

- **맥락 단절** — 질문마다 창이 나뉘어 주제 → 가설 → 분석 → 논문으로 이어지는 맥락이 끊긴다.
- **자기 검증의 편향** — 답을 만든 모델이 스스로 검증하면 같은 편향을 공유한다.
- **인용 환각** — 존재하지 않는 논문을 그럴듯하게 인용한다.
- **대필 위험** — 연구자의 해석 없이 완성된 글을 내놓아 연구윤리·학습 효과를 해친다.

1.2 설계 원칙

원칙	구현
하나의 창구, 여러 전문가	연구자는 캐릭터 '키스토' 한 명과 대화한다. 뒤에서는 역할별 연구원 모듈이 일하고, 페르소나 레이어가 모든 답변을 한 사람의 말투로 통일한다.
다른 회사 모델의 검토	결과를 실제로 만든 모델과 회사가 다른 모델을 검토위원 1순위로 코드가 강제 배정한다 (알고리즘 2).
근거는 코드가 보증	참고문헌 목록은 LLM이 쓰지 않고, 실제 검색 결과로 코드가 만든다 (알고리즘 3).
해석은 연구자의 몫	초안 요청에 먼저 연구자의 해석을 묻고, 그 해석이 저장된 뒤에만 초안을 쓴다 (알고리즘 5).
보이는 진척은 실데이터로	로드맵·연구실·대시보드의 모든 수치는 저장된 기록에서 계산한다 (알고리즘 6).
패턴 차용, 연동 주장 없음	Google AI Co-Scientist, FutureHouse Kosmos의 멀티에이전트 패턴을 참고해 자체 재구현했다. 두 서비스는 공개 API가 없으며 연동하지 않았다.

2 시스템 구조

로컬 PC에서 FastAPI 백엔드와 Electron 위젯이 함께 실행된다. 연구자의 메시지는 개인정보 가림 → 의도 라우터 → 연구 맥락 주입 → 연구원 모듈을 거친 뒤, **페르소나 다듬기와 교차검증을 병렬로** 받아 한 번에 응답된다. 모든 결과는 주제별 스레드에 기록되고, 그 기록에서 로드맵·연구실·대시보드가 계산된다.

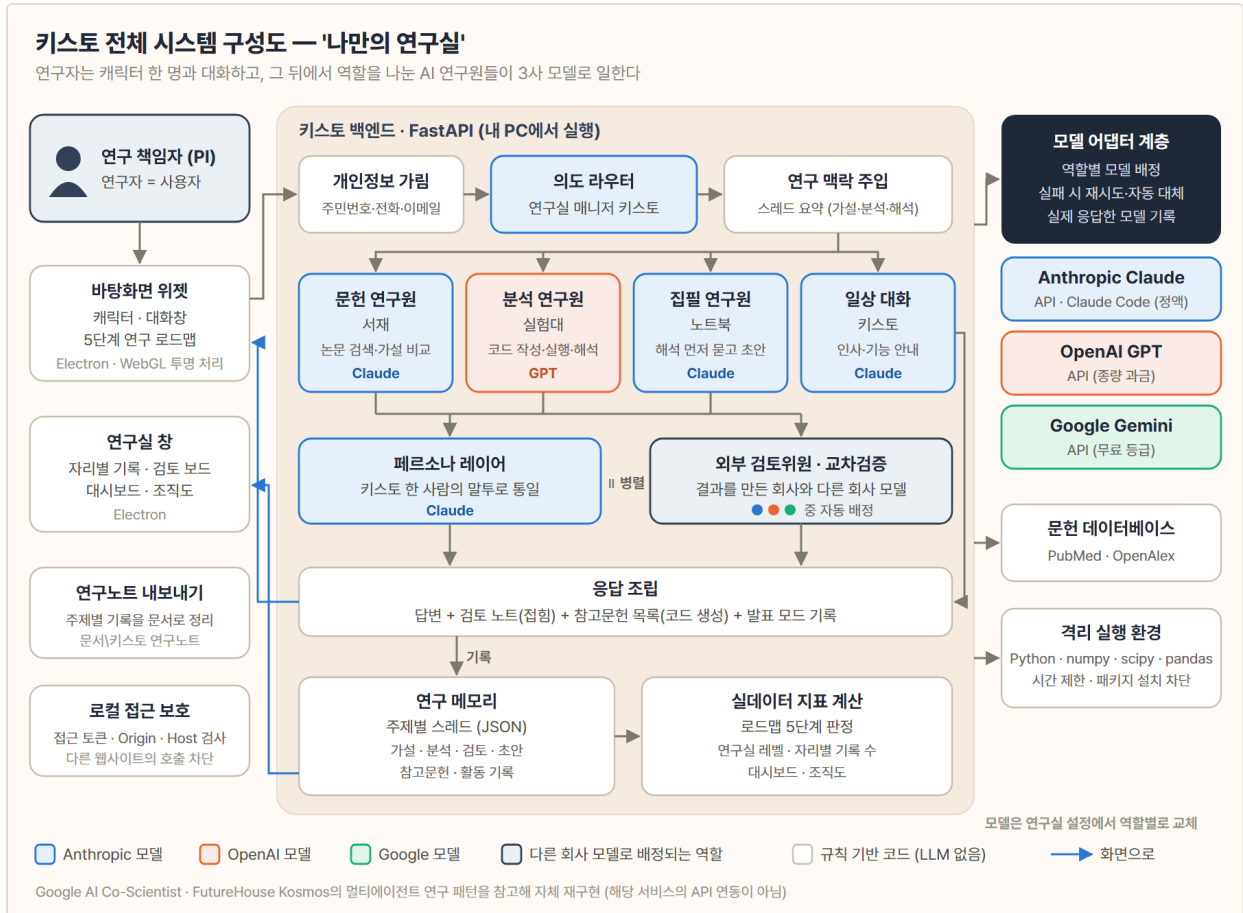


그림 1. 전체 시스템 구성도

구성원	자리	기본 모델	하는 일	구현
연구실 매니저	위젯	Claude	의도 분류, 말투 통일	orchestrator/router.py , persona.py
문헌 연구원	서재	Claude	실제 논문 검색, 선행 연구 요약, 후보 가설 비교	modules/research_coach.py , literature.py
분석 연구원	실험대	GPT	분석 코드 작성 → 격리 실행 → 자동 수정	modules/analysis_partner.py , sandbox/
집필 연구원	노트북	Claude	해석 질문 → 해석을 반영한 초 안	modules/writing_coach.py
외부 검토위원	검토 보드	다른 회사	과잉해석·통계 오류·인용 불일 치 지적	modules/verifier.py , config.py

역할별 모델은 연구실 화면의 '연구원 명단'에서 연구 책임자가 바꿀 수 있고(GET/PUT /lab/roles), 키가 없는 모델은 자동으로 다른 모델이 대신한다. 모델 어댑터는 Claude API, Claude Code(정액 구독), OpenAI, Gemini 4종이다.

3 알고리즘 1 · 의도 라우팅과 상태 기반 모듈 선택

구현

backend/orchestrator/router.py · backend/main.py

문제. "이걸로 분석해 줘", "논문에 녹여 줘"처럼 연구 대화의 상당수는 앞선 맥락이 있어야 의미가 정해진다. LLM에게 모듈 선택까지 맡기면 파이프라인 단계를 건너뛰거나 되돌아가는 오류가 생긴다.

방법. LLM은 5개 의도 중 하나와 (필요하면) 새 주제 이름·영어 검색어만 JSON으로 분류한다. 어느 연구원에게 보낼지는 **스레드**에 저장된 **연구 상태**에 대한 규칙이 결정한다. JSON 해석에 실패하면 가장 안전한 `topic_explore` 로 처리한다.

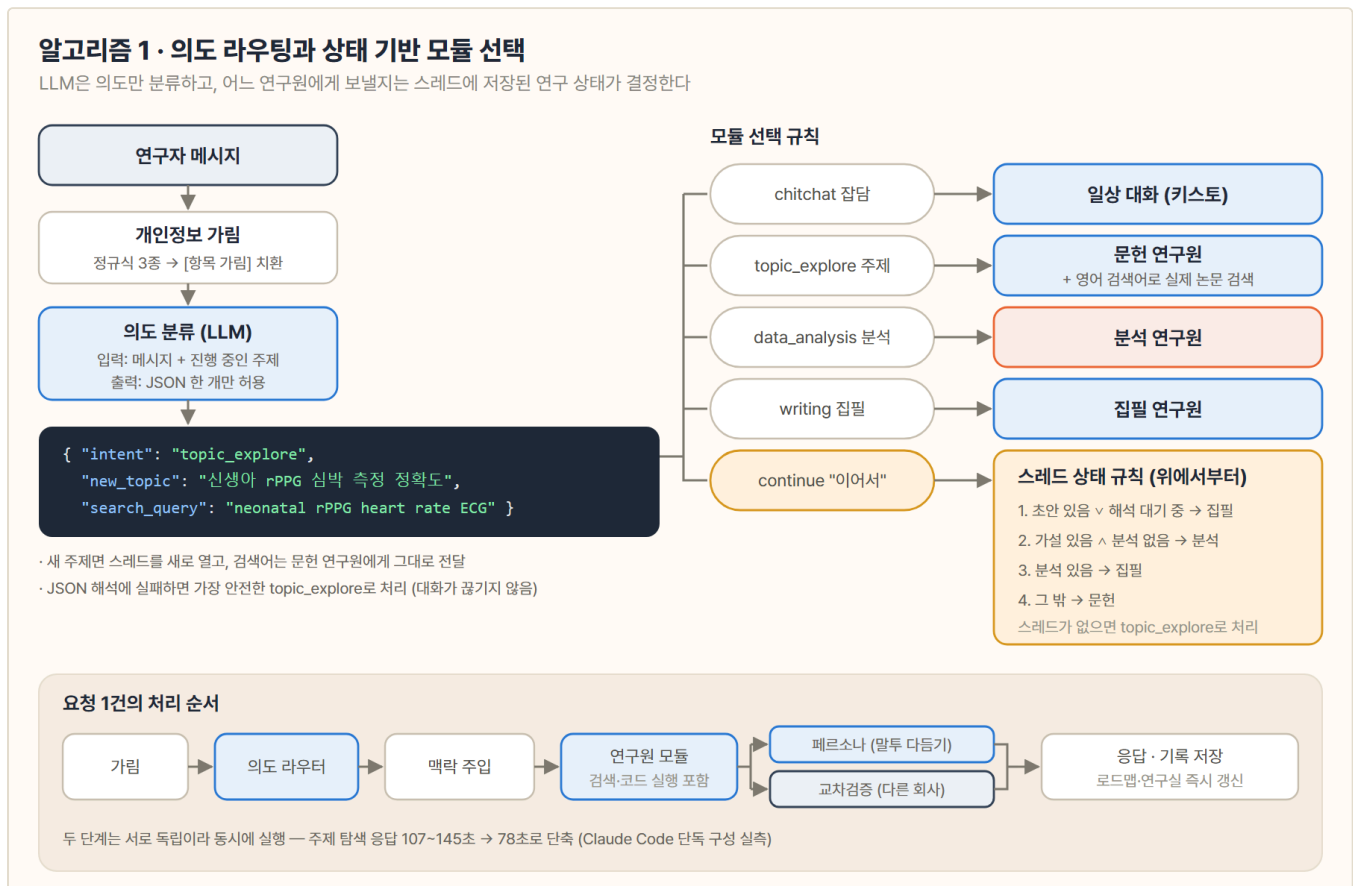


그림 2. 의도 분류와 상태 규칙, 요청 1건의 처리 순서

확인한 효과

테스트 중 "분석 이후 이어서" 요청이 문헌 연구원으로 되돌아가던 문제를 상태 규칙 3번으로 해결했다. 페르소나와 교차검증은 서로 독립이라 병렬로 실행해, 주제 탐색 응답을 107~145초에서 78초로 줄였다 (Claude Code 단독 구성 측정).

4 알고리즘 2 · 이중 모델 교차검증 배정과 자동 대체

구현

`backend/config.py ( get_verifier_adapter , FallbackAdapter ) · backend/modules/verifier.py`

문제. 같은 모델이 자기 결과를 검증하면 같은 편향을 공유한다. Google AI Co-Scientist도 같은 모델 계열(Gemini) 안에서 생성과 검증을 한다. 또 외부 API는 실제로 자주 막힌다 — 9월 21일 첫 실행에서 요청 폭주(503)와 모델 없음(404)을 겪었다.

방법. 결과를 실제로 만든 모델 p^* 의 회사를 기준으로 후보를 세 층으로 정렬한다: 다른 회사 → 같은 회사 다른 모델 → 자기 검토. Claude API와 Claude Code는 같은 회사로 본다. 각 후보는 공통 Fallback 절차로 호출되며, 일시적 오류는 4초·10초 대기 후 재시도하고 그 밖의 오류는 다음 후보로 넘어간다. 검토를 건너뛰지는 않으며, 어느 층에서 검토했는지 기록에 남긴다.

알고리즘 2 · 이중 모델 교차검증 배정과 자동 대체

결과를 실제로 만든 모델의 회사를 빼고 검토위원을 고른다. 호출이 실패해도 검토는 건너뛰지 않는다

검토위원 후보 순서

```
입력  $p^*$  = 결과를 실제로 만든 모델,  $p$  = 사용 가능한 모델
vendor(Claude API) = vendor(Claude Code) = Anthropic
vendor(GPT) = OpenAI, vendor(Gemini) = Google

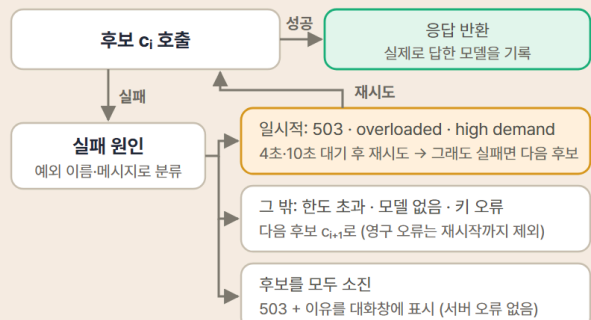
L ← 중복 제거( [사용자 지정] + [Gemini, GPT,
                Claude API, Claude Code] + P )

 $C_1 \leftarrow \{ p \in L \mid \text{vendor}(p) \neq \text{vendor}(p^*) \}$ 
# 1순위: 다른 회사

 $C_2 \leftarrow \{ p \in L \mid \text{vendor}(p) = \text{vendor}(p^*), p \neq p^* \}$ 
# 2순위: 같은 회사 다른 모델 (기록에 표시)

 $C_3 \leftarrow \{ p^* \}$  # 최후: 자기 검토 (기록에 표시)
return Fallback(  $C_1 \oplus C_2 \oplus C_3$  )
```

Fallback 실행 — 모든 LLM 호출에 공통



2026-09-21 실제 실행 기록 — 검토 3건 모두 다른 회사 모델이 수행

① 주제 탐색 · 선행 연구 요약



② 데이터 분석 · 해석



③ 논문 서론 초안



- 검토 입력: 결과 본문 + (있으면) 인용 번호가 붙은 근거 논문 초록 → 과잉해석 · 통계 오류 · 재현성 · 인용 불일치를 지적
 - 검토 결과는 답변에 섞지 않고 '검토 노트'로 분리 전달, 검토 보드에 만든 회사 · 검토 회사와 함께 기록
 - 연구 책임자가 검토위원 모델을 같은 회사로 지정해도, 다른 회사 모델이 있으면 그쪽을 우선한다
- 비교: Google AI Co-Scientist는 같은 모델 계열(Gemini) 안에서 검증 — 키스토는 회사가 다른 모델만 1순위로 둔다

그림 3. 검토위원 후보 순서, Fallback 실행, 실제 실행 기록

왜 '실제로 답한 모델'을 기록하는가

폴백이 일어나면 예정된 모델과 실제로 답한 모델이 달라진다. 예정 기준으로 기록하면 교차검증 비율이 사실과 어긋난다. 그래서 호출이 성공한 순간의 모델을 추적(`tracing.note_answer`)해 생성자·검토자 판정의 근거로 쓴다.

5 알고리즘 3 · 근거 고정 인용

구현

```
backend/literature.py · modules/research_coach.py · modules/writing_coach.py ·  
memory/store.py ( add_references )
```

문제. LLM은 존재하지 않는 논문을 그럴듯한 서지 형식으로 만들어 낸다. 프롬프트로 "지어내지 마라"라고만 하면 막을 수 없다.

방법. 생성 영역과 결정 영역을 분리한다. 라우터가 뽑은 영어 검색어로 PubMed(생의학 우선)와 OpenAlex(전 분야 보완)를 검색해 DOI·제목 기준으로 중복을 없애고 최대 6편을 고른다. 논문에는 스레드 전체에서 유지되는 번호 [n]을 붙인다. LLM은 이 번호로만 인용할 수 있고, 참고문헌 목록은 본문에서 실제로 쓰인 번호만 골라 **코드가** 검색된 서지 그대로 붙인다. 검토위원은 인용 문장과 해당 초록을 대조한다.

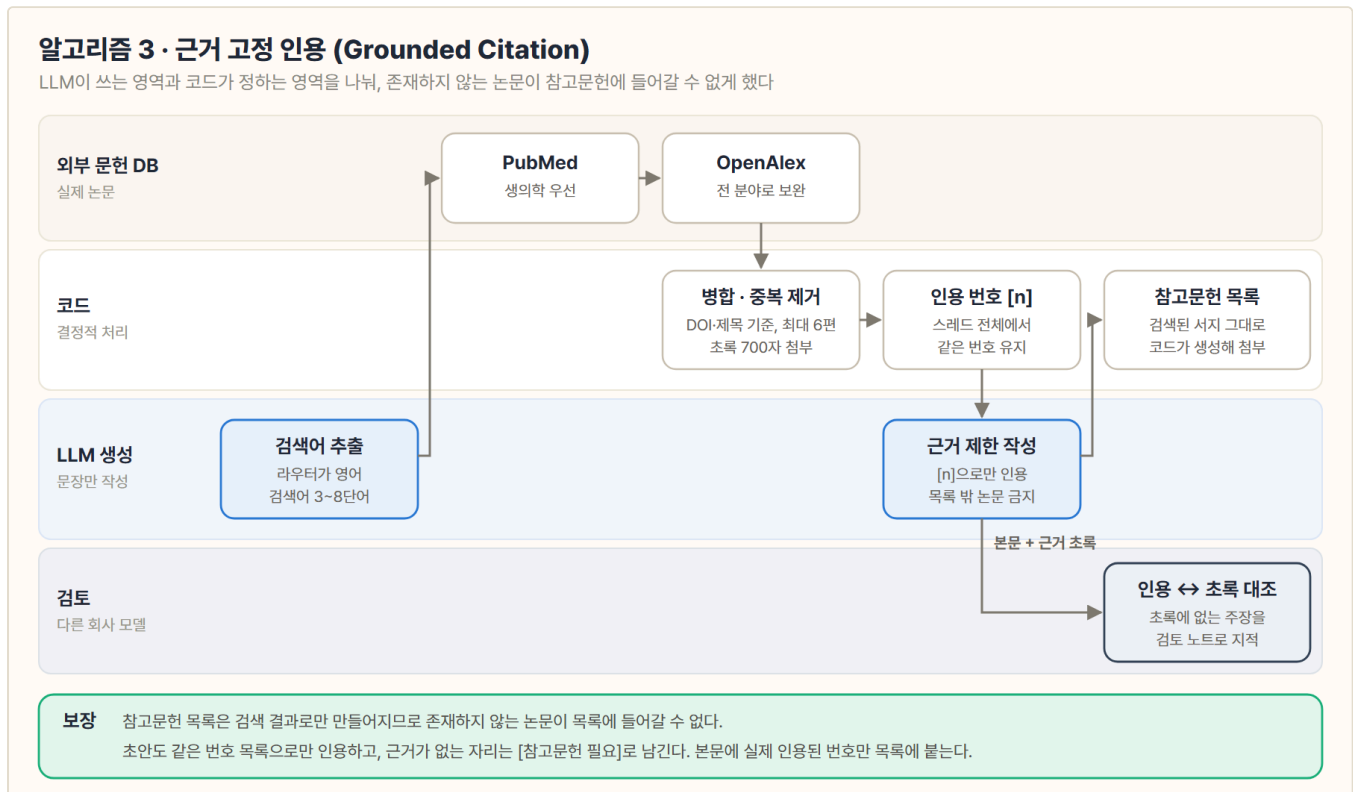


그림 4. 외부 DB · 코드 · LLM · 검토 4개 영역으로 나눈 인용 파이프라인

확인한 동작 (2026-09-21)

PubMed에서 실제 논문 6편을 찾아 [번호]로 인용했고, 근거 논문이 없는 문장("미숙아 피부 손상")에는 인용을 지어내지 않고 [참고문헌 필요]로 남겼다. 검토위원(GPT)은 인용 [6]이 해당 문장의 근거로 맞지 않을 가능성을 지적했다.

6 알고리즘 4 · 자가 수정 분석 루프

구현

backend/modules/analysis_partner.py · backend/sandbox/executor.py

문제. 생성된 분석 코드는 한 번에 돌지 않는 경우가 많고, 수치만으로는 연구 질문에 대한 답이 되지 않는다.

방법. 코딩에 강한 모델(GPT)이 코드를 쓰고, 임시 폴더에서 첨부 파일의 복사본만으로 격리 실행한다. 실패하면 코드와 에러 메시지를 넘겨 최대 2회 스스로 고친다. 해석은 문헌 연구원과 같은 모델이 **원래 요청 + 연구 맥락(가설) + 코드 + 출력**을 함께 보고 가설을 지지하는지 반박하는지 판단한다. 결과는 스레드에 기록되고 다른 회사 모델의 검토를 받는다.

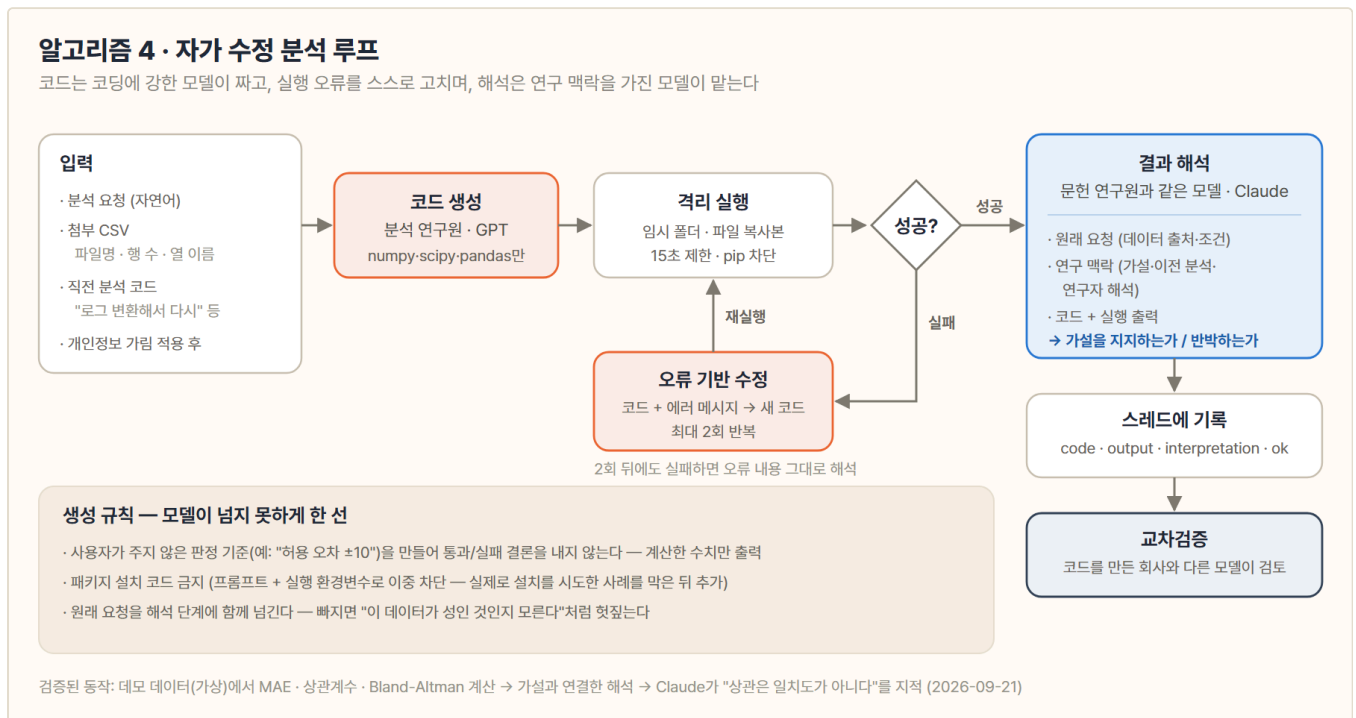


그림 5. 코드 생성 → 격리 실행 → 오류 기반 수정 → 맥락 해석 → 기록·검증

확인한 효과

가상 데이터(n=12)에서 MAE·상관계수·Bland-Altman을 계산하고 가설과 연결해 해석했다. 검토위원(Claude)은 GPT가 만든 해석에서 "피어슨 r은 일치도가 아니라 연관성 지표"라는 통계적 과대해석을 지적했다.

7 알고리즘 5 · 대필하지 않는 집필 프로토콜

구현

backend/modules/writing_coach.py · backend/main.py

문제. "초안 써 줘"에 곧바로 완성된 글을 주면 연구자는 자기 결과를 해석하는 연습을 건너뛰게 되고, 연구윤리상 대필이 된다.

방법. 2단계 상태 기계로 구현했다. 첫 요청(S0 → S1)에는 분석 핵심만 짚고 해석을 묻는 질문 2개 이내로 답한다. 연구자의 답(S1 → S2)이 오면 그것을 반드시 반영해 초안을 쓰고, 해석은 스레드에 저장되어 이후 모든 모듈의 맥락과 연구노트에 들어간다. 해석 질문은 페르소나를 거치지 않는다 — 말투를 다듬는 단계가 질문에 스스로 답해 버리는 현상을 실제로 겪었기 때문이다. 초안 역시 말투 변환 없이 원문 그대로 전달한다.

알고리즘 5 · 대필하지 않는 집필 프로토콜

연구자의 해석이 저장되기 전에는 초안이 만들어지지 않는 2단계 상태 기계

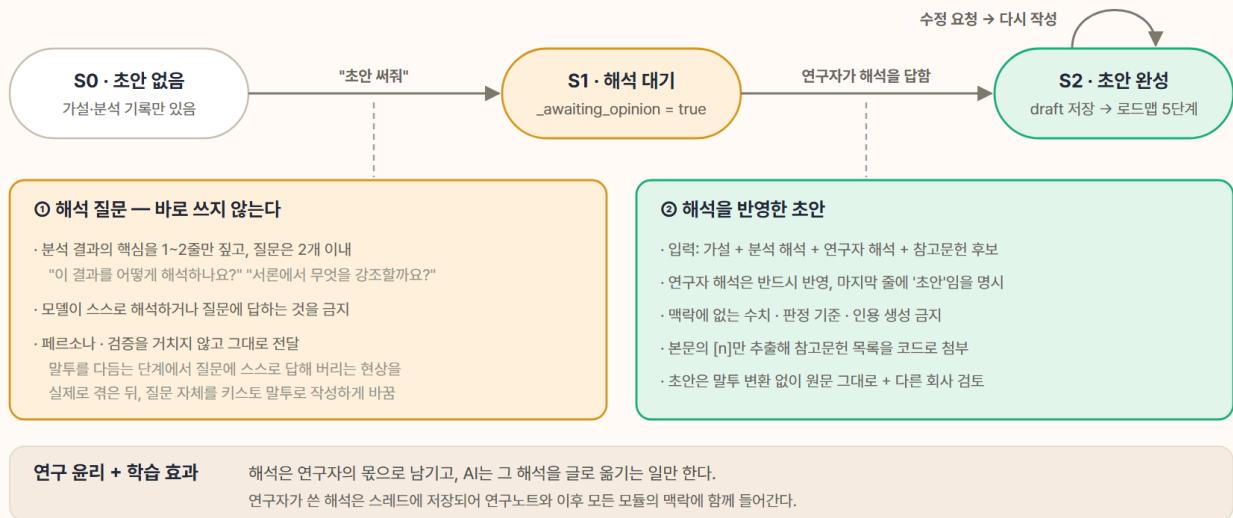


그림 6. 해석 대기 상태를 거쳐야만 초안이 만들어지는 상태 기계

확인한 동작

데모 3단계에서 집필 연구원은 초안을 쓰지 않고 해석을 먼저 물었고(12초), 4단계 초안은 연구자가 답한 "보조 수단으로서 가능성, 다만 n=12·통제 환경 한계" 해석을 반영했다.

8 알고리즘 6 · 연구 메모리와 실데이터 기반 게임화

구현

backend/memory/store.py (thread_brief , thread_progress) · backend/lab.py · widget/lab.js

문제. 게임 요소가 실제 진척과 무관하게 움직이면 장식에 그치고 신뢰를 잃는다. 또 모듈마다 현재 메시지만 받으면 앞선 연구를 잊는다.

방법. 주제 하나를 스레드 하나로 저장하고, 그 요약본(thread_brief)을 모든 모듈에 주입한다. 기록이 없을 때는 맥락을 넘기지 않는다 — 빈 맥락을 주면 모델이 "지난번에 말씀드린 것처럼"이라며 없는 대화를 지어낸다. 로드맵 5단계는 저장된 필드에 대한 술어로 판정하고, 연구실의 책·시험관·메모지, 레벨, 대시보드 지표도 같은 기록에서 계산한다.

알고리즘 6 · 연구 메모리와 실데이터 기반 게임화

로드맵·연구실·대시보드의 모든 수치는 스레드에 저장된 기록에서 계산된다 — 기록이 없으면 채워지지 않는다

연구 스레드 (주제 1개)

```
thread {  
  topic, created_at  
  hypotheses[] # 문헌 연구원  
  references[] # 검색된 논문  
  files[] # 첨부 CSV  
  analysis[] # 코드·출력·해석  
  verifications[] # 분석 검증  
  reviews[] # 만든·검토 회사  
  interpretation # 연구자 해석  
  draft, draft_at  
  turns[], activity[] # 시간·모델  
}
```

모든 모듈에 요약본(thread_brief) 주입

로드맵 5단계 판정 (술어)

- 1 주제 잡기
← 스레드가 존재
- 2 가설 세우기
← hypotheses ≠ ∅
- 3 분석 돌리기
← analysis ≠ ∅
- 4 교차검증
← verifications ≠ ∅
- 5 초안 쓰기
← draft 저장됨

진행률 = 달성 단계 수 / 5 (예: 3/5 = 60%)

연구실 화면

책 = 찾은 논문 수 (서재)
시험관 = 분석 횟수 (실험대)
메모지 = 검토 기록 (검토 보드)

연구실 레벨

달성한 단계 1개 = 별 1개
별 5개마다 레벨 +1
새내기 연구실 → ... → 명예의 전당

대시보드

이중 검토 비율 =
만든 회사 ≠ 검토 회사인 검토 / 전체 검토
연구원별 작업 시간 = activity 단계 합
회사별 호출 = 실제로 답한 모델 기준

'주제별 대화 풀더' 대신 진행 단계를 보여준다 — 같은 기록이 대화 맥락, 로드맵, 연구실 그림, 대시보드, 연구노트를 동시에 만든다

그림 7. 스레드 스키마 → 단계 판정 술어 → 연구실·레벨·대시보드

로드맵 5단계(주제 잡기 → 가설 → 분석 → 교차검증 → 초안)는 AutoResearch 5단계 프레임워크에 대응한다. "주제별 대화 풀더" 대신 진행 단계를 보여주는 방식을 택해 Claude·ChatGPT Projects와 구별된다.

9 실측 결과

2026-09-21 `scripts/e2e.py` 로 데모 시나리오(신생아 카메라 비접촉 심박 측정 vs 심전도 일치도)를 Anthropic·OpenAI·Google 3사 모델을 동시에 연동해 실행한 기록이다. 시간은 키스토 시스템의 동작을 켜 것이며, 분석 수치는 가상 데이터로 계산했다.

9.1 단계별 결과

단계	만든 모델	검토 모델	이종 검증	소요
① 주제 탐색 (실제 논문 6편 검색)	Claude	Gemini	O	82초
② 데이터 분석 (코드 작성·실행·해석)	GPT	Claude	O	90초
③ 해석 질문 (대필 금지)	Claude	— (검토할 결과 없음)	—	12초
④ 서론 초안 (연구자 해석 반영)	Claude	GPT	O	107초
합계	3사 모두 참여		3 / 3	292초

같은 시나리오의 소요 시간은 435초(순차 실행) → 297초(검토·말투 병렬화) → 292초(3사 구성)로 줄었다.

9.2 검토위원이 잡아낸 오류

검토	지적 내용
Claude → Gemini	근거 논문의 NIRS(접촉식)를 비접촉 기술로 잘못 설명, 초록에 이미 있는 기법을 연구 공백처럼 서술
GPT → Claude	피어슨 r은 일치도가 아니라 연관성 지표 — $r=0.95$ 로 측정 일치를 주장하는 과대해석, 편향의 신뢰구간 누락
Claude → GPT	①의 NIRS 오류가 초안에 남아 있음을 다시 지적, 인용 [6]이 해당 문장의 근거로 맞지 않을 가능성

9.3 오류 탐지 평가와 비용 구조

- 통계·해석 오류를 심어 둔 결과 10건(상관≠일치, 다중비교 미보정, 인과 오해석, 초록에 없는 인용 등)과 정상 결과 2건으로 검토위원을 평가한 결과 **10건 모두 탐지, 오탐 0건** (`scripts/verifier_eval.py`).
- 데모 1회 기준 모델 호출의 80%, 생성 토큰의 84%를 정액 구독·무료 등급 구간에서 처리했다. 종량 과금은 분석(GPT) 구간만 발생한다.

10 보안·개인정보, 한계와 향후 과제

10.1 보안·개인정보

- **개인정보 자동 가림** — 메시지와 첨부 파일에서 주민등록번호·전화번호·이메일을 외부 모델로 보내기 전에 가린다 (`privacy.py`). 영유아 연구처럼 민감한 데이터에서 실수로 붙여 넣은 식별정보가 나가지 않게 하는 최소한의 장치다.
- **로컬 서버 보호** — 접근 토큰, Origin 허용 목록, Host 검사로 다른 웹사이트가 로컬 백엔드를 호출해 기록을 읽거나 분석 코드 실행을 유도하는 경로를 막았다.
- **실행 격리** — 생성 코드는 임시 폴더에서 파일 복사본만으로 실행하고, 시간 제한과 패키지 설치 차단을 둔다.
- **키 관리** — API 키는 `.env` 에만 두고 저장소에 넣지 않는다.

10.2 한계

있는 그대로 적는 한계

- 실행 격리는 프로토타입 수준(하위 프로세스 + 임시 폴더)이다. 여러 사용자에게 배포하려면 컨테이너 격리가 필요하다.
- 오류 탐지 평가(10/10)는 Claude 단독으로 측정했고, 사례가 쉬워 천장 효과가 있다. 회사별 차이를 보려면 교란 변수·단위 착오·자유도 오류 같은 더 미묘한 사례가 필요하다.
- 검토 노트는 자동 수정이 아니라 연구 책임자에게 보이는 노트다. 그래서 앞 단계의 지적(NIRS)이 초안까지 이어진 사례가 있다 — 최종 판단을 연구자에게 두는 설계의 결과이며, 이전 검토 지적을 다음 단계에 넘기는 것이 개선 후보다.
- 분석 결과 수치는 시연용 가상 데이터로 계산한 값이다.

10.3 향후 과제

- 이전 검토 지적을 다음 단계(초안)의 입력으로 전달
- 보안 요구가 높은 과제를 위해 특정 역할만 사내·로컬 모델로 교체
- 어려운 오류 사례를 추가해 회사별 검토 성능 비교

10.4 참고한 선례

- Google AI Co-Scientist (2025) — 생성·토론·진화 에이전트 기반 가설 생성
- FutureHouse Kosmos — 장기 자율 연구 에이전트
- Sakana AI Scientist v1/v2 — 아이디어 → 실험 → 논문 → 리뷰 자동화
- AutoResearch 5단계 프레임워크 — 로드맵 5단계의 근거